

Depth, Breadth, or Modularity? A Parameter-Matched Benchmark of Hybrid Neural Architectures for Inference Latency and Accuracy

Aryan Kulkarni

Bellarmino College Preparatory

San Jose, CA, USA

aryanamol10@gmail.com

Abstract—Architecture comparisons often use parameter counts or FLOPs as proxies for speed, but deployed systems are judged by latency. We benchmark six small neural architectures, three single-paradigms (sequential, parallel, modular) and three hybrids combining depth, breadth, and modularity, all held to 385 parameters ($\pm 0.5\%$) and evaluated on three regression tasks with five seeds each. Other studies suggested a $1.62\times$ speedup for modular networks. An examination traced it to uninformative inputs, in which we present it as a case study in benchmarking pitfalls. In the corrected study, every topology reaches the noise floor on simple tasks. On a composition task, a staged modular hybrid lowers test error by 43% relative to a sequential baseline ($p < 0.05$) and meets in far fewer epochs at $3.3\times$ the single-sample latency. Across models, single-sample latency is almost perfectly predicted by tensor operations per forward call ($R^2 = 0.98$) rather than by parameter count, and at large batch sizes the ranking inverts in favor of designs. Smaller architectures should therefore be compared at a deployment batch of size, with operator counts reported alongside parameters.

Index Terms—neural network architecture, hybrid architectures, modular networks, inference latency, benchmarking, efficient machine learning, reproducibility

I. INTRODUCTION

Choosing a neural architecture means choosing how to spend constant budgets of parameters and compute. The same budget can buy depth (precision in layers), breadth (parallel bindings), or modularity (sub-networks that each handle part of a problem). Modern systems increasingly mix all three of these types: multi-branch blocks [1], [2], shared trunks with task heads [3], and mixtures of experts [4]. Theory says depth and width trade off in representational power [5], [6]. People working on this, however, also care about a different concept: *how fast does the model answer?*

Latency is notoriously hard to predict from the numbers that architecture papers usually report. Parameter count and FLOPs can disagree with the measured speed and can create a reverse in the model’s rankings [7], [8]. The gap is widest for small models, such as those in embedded control, sensor fusion, or TinyML [9], where fixed per-operator overhead, rather than arithmetic, can dominate run time [10]. Benchmarks of such models are also easy to get wrong. Framework choice, seed variance, and evaluation shortcuts can each produce differences larger than the effect being studied [11], [12].

This paper asks: *at a fixed parameter budget, does the arrangement of parameters (sequential, parallel, modular, or a hybrid) change accuracy and inference latency, and what predicts the latency?* We answer with a controlled benchmark of six architectures at 385 parameters ($\pm 0.5\%$) on three synthetic regression tasks of increasing structural complexity. Synthetic tasks let us know the irreducible error exactly and let us design one task whose compositional structure a modular model could benefit from.

Contributions.

- A *parameter-matched benchmark* of three single-paradigm and three hybrid MLP architectures, with five seeds, held-out evaluation, and null control (two functionally identical models), released with all code and raw results.
- An *audited pilot study* showing how a cross-system comparison produced a false $1.62\times$ “modular speedup”, and three low-cost practices that catch such errors.
- Evidence that *structure helps only when the task has structure*. All topologies tie on simple targets, but a staged modular hybrid reduces error on a composed target by 43% and mixes much faster.
- A latency analysis showing that, for small models, *single-sample latency is predicted by operator count* ($R^2 = 0.98$) rather than parameters or MACs, and that the latency ranking *depends on batch size*.

Section II reviews related work, Section III describes the setup, Section IV audits the pilot, Section V presents results, and Section VI discusses implications and limitations.

II. RELATED WORK

Depth versus breadth. A single hidden layer of sufficient width can give an approximation of any continuous function [13], [14], yet depth-separation results show functions that deep networks represent far more efficiently than shallow ones [5], [15], and width has its own limits [6]. These results concern represented capacity. We ask the logical question that follows from them, which has us answer at a fixed, small parameter budget, which sequence trains to smallest amount of error, and which one runs fastest?

Multi-branch and shared-trunk designs. Parallel branches are pivotal to Inception [1] and ResNeXt [2], and wide

remaining networks trade depth for width [16]. A shared trunk with several heads is the joint learning pattern of multi-task learning [3], [17], and averaging several predictions underlies deep coordinations from different aspects [18]. Our Parallel, H-A, and H-C models are small examples of these ideas.

Modular networks. Mixtures of experts route inputs to specific sub-networks [19] and scale to very large models [4]. Neural module networks combine modules to mirror the compositional structure of a query [20], and a recent survey notes the advantages and costs of modularity [21]. Whether neural networks utilize compositional structure at all is an open question [22]. H-B tests the simplest version of this hypothesis, showing one module per stage of a known composition joined by dense connections [23], [24].

Measuring efficiency. Parameter count and FLOPs are known to be poor delegates for latency: ShuffleNet V2 derived architectures from directly measured speed [7], and Dehghani et al. showed that the choice of cost indicator can reverse model rankings [8]. Hardware-aware architecture search therefore optimizes measured latency [25]–[27]. Standard benchmarks such as MLPerf Inference [28], MLPerf Tiny [9], and DAWNBench [29] fix the measurement rules, and surveys of efficient processing explain where time goes in DNN inference [10], [30]. Efficient architectures for restricted devices [31] motivate studying very small models. Benchmarks are also fragile. Framework choice alone changes measured speed [11], seed and data-split change can exceed the effects being claimed [12], [32], and empirical ML has a history of overstated conclusions [33]. Our pilot study (Section IV) shows a solid small-scale example of the first two problems.

Positioning. Prior latency studies target large vision or language models on accelerators. We study the opposite end, sub-kilobyte MLPs on a single CPU thread, where overhead is visible in isolation. Unlike capacity comparisons, we keep parameters constant and change only structure.

III. EXPERIMENTAL SETUP

A. Synthetic Regression Tasks

We generate $N = 5,000$ evenly spaced inputs $x \in [0, 10]$ and three targets of increasing structural complexity (Fig. 1), each with additive Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$:

$$y_{\text{lin}} = 2x + \varepsilon, \quad \sigma = 1, \quad (1)$$

$$y_{\text{curv}} = 0.5x^2 + \sin x + \varepsilon, \quad \sigma = 2, \quad (2)$$

$$y_{\text{chain}} = s_a + \sin(s_a) + 0.5 \operatorname{sgn}(\sin x) + \varepsilon, \quad \sigma = 0.5, \quad (3)$$

where $s_a = x \cos(0.5x)$. The *chained* target is a composition of three stages: a dynamic ramp, a non-linear instance applied to this ramp, and a discontinuous square wave. This was designed so for a model whose structure mirrors the stages could, in principle, utilize it. Because the noise is known, each task has an indivisible test MSE of σ^2 (1, 4, and 0.25, respectively), which we use as a complete reference. Inputs are scaled to $[0, 1]$; targets are left in their original units. A single random 80/20 train/test split (seed 42) is shared by every model and task, giving 4,000 training and 1,000 test points.

TABLE I
ARCHITECTURE COST PROFILE. DEPTH = LINEAR LAYERS ON THE LONGEST INPUT-OUTPUT PATH. OPS = PYTORCH TENSOR OPERATIONS DISPATCHED PER FORWARD CALL; KERN. = ATEN KERNELS EXECUTED. LATENCY = MEDIAN SINGLE-THREAD CPU INFERENCE TIME OVER 10 SHUFFLED REPEATS.

Model	Params	MACs/			Latency (μ s)		
		sample	Depth	Ops	Kern.	b=1	b=4096
Sequential	385	256	2	3	22	11.7	196
Parallel	385	256	2	6	35	16.1	314
Modular	385	256	2	3	22	10.0	193
H-A	385	336	3	10	59	31.7	213
H-B	385	286	6	13	70	38.2	322
H-C	387	350	3	8	50	25.0	156

B. Architectures

All six models (Fig. 2) are one-input, one-output multilayer models with ReLU activations [34], each with exactly 385 trainable parameters, so that differences in accuracy or speed cannot be given to capacity. The one exception is H-C: no width pair for two-head wiring totals 385, so it uses the nearest configuration, 387 (+0.5%). Table I lists their parameter counts, multiple accumulated operations (MACs) per sample, critical-path depth (linear layers between input and output), tensor operations given per forward call, and low-level (ATen) kernels those loops execute.

Single-paradigm baselines. *Sequential* is a 1–128–1 network. *Parallel* feeds the input to two 64-unit branches whose activations are concatenated before a linear read-out, in the spirit of multi-branch designs such as Inception [1]. *Modular* wraps a hidden layer and read-out in a self-contained module. With 128 hidden units it calculates exactly the same function class as Sequential, and under the same seed it initializes to the same weights. We keep it as a *control*: any measured difference between Sequential and Modular is implementation overhead or timing noise, never architecture.

Hybrids. **H-A** (*parallel of sequentials*) runs two 2-layer, 12-unit stacks side by side, combining depth and breadth. **H-B** (*staged modular chain*) has three sub-networks M_A , M_B , M_C (hidden widths 33, 32, and 31, chosen to total exactly 385 parameters) that mirror the three stages of the chained target: M_B receives (x, a) , M_C receives (x, a, b) , and the output is $a + b + c$. This gives densely connected skip links in the spirit of DenseNet [24] and an additive output similar to residual learning [23]. **H-C** (*trunk + parallel heads*) shares a 2-layer trunk (20 and 15 units) between two linear heads whose outputs are averaged, the hard-parameter-sharing pattern from multi-task learning [3]. No set of architectures can match parameters, MACs, and operation count at once. Fixing parameters forces the other two to vary, and that difference is what lets us test which cost measure actually predicts latency.

C. Training Protocol

Every model is implemented and trained in PyTorch [35] with one synchronized training loop. We use Adam [36]

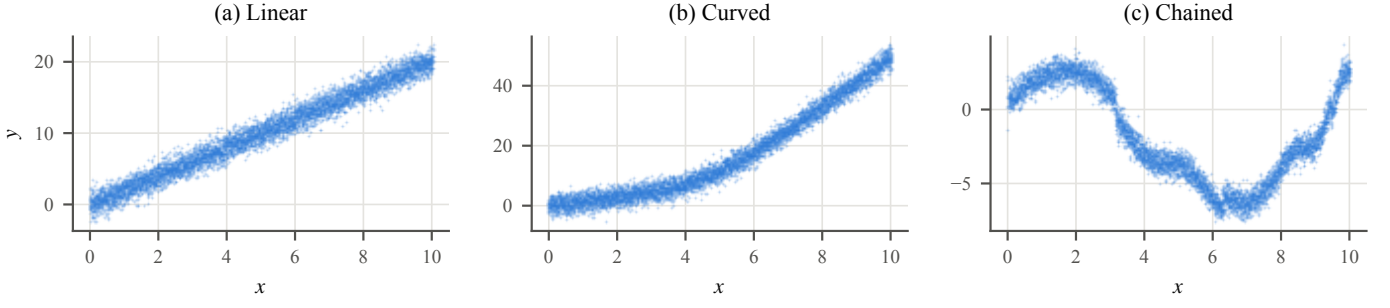


Fig. 1. The three synthetic regression tasks ($N = 5,000$). (a) linear with $\sigma = 1$; (b) quadratic plus sinusoid with $\sigma = 2$; (c) a three-stage composition containing a square-wave discontinuity, $\sigma = 0.5$.

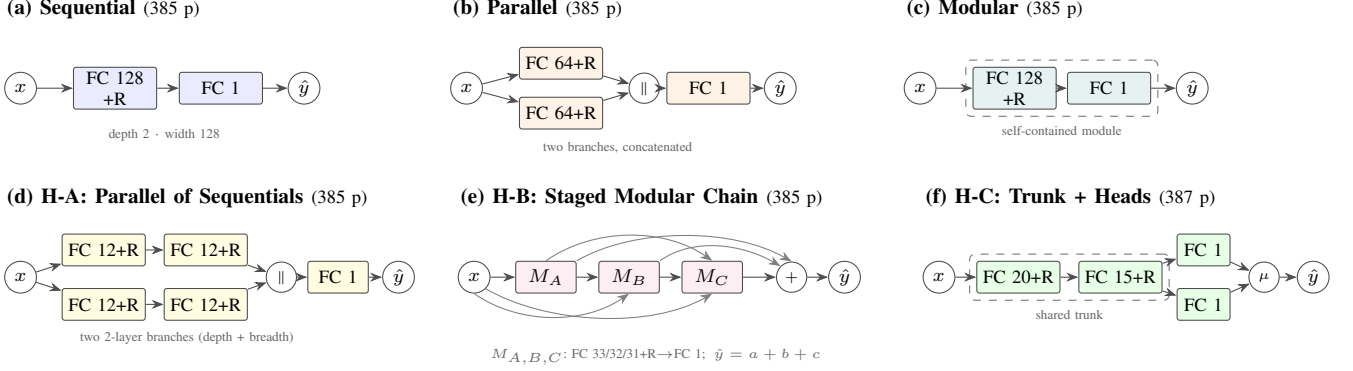


Fig. 2. The six parameter-matched architectures (385 trainable parameters; 387 for H-C). Top row: the three single-paradigm baselines from the pilot study. Bottom row: hybrids combining depth, breadth, and modularity. FC n = fully connected layer with n outputs, R = ReLU, $\parallel$ = concatenation, μ = mean. Note that (a) and (c) compute the same function class; they differ only in how the code is organized, which lets us use them as a control pair for implementation overhead (Section IV).

(learning rate 10^{-3} , $\beta = (0.9, 0.999)$, $\epsilon = 10^{-7}$, matching the Keras default used in the pilot), mean-squared-error loss, mini-batches of 32 with per-epoch shuffling, and 100 epochs (12,500 optimizer steps). Each (model, task) pair is trained with five seeds (0–4) that control starting and batch order. We report mean $\pm$ standard deviation across seeds, following recommendations to account for training variance in benchmark comparisons [12].

D. Metrics

- **Test MSE** and R^2 on the 1,000 held-out points after the final epoch.
- **Convergence epoch**: the first epoch whose test MSE is within 5% of that run’s best.
- **Training cost**: clock training time divided by optimizer steps (ms/step).
- **Inference latency**: median of 50 forward passes over the full test set (batch 1,000).

Significance against the Sequential baseline uses Welch’s two-sample t -test on test MSE ($n = 5$ per arm, $p < 0.05$). During the first pass, the control pair flagged CPU contention on the linear task: two models’ step times nearly doubled while their accuracy was unchanged. We re-timed that task in full. Accuracy was bit-identical across both passes, and only the clean timings are reported.

E. Latency Sweep

Real deployments rarely run inference at exactly the batch size used for evaluation, so we also time every model at batch sizes $\{1, 8, 64, 512, 1000, 4096\}$ with 1 and 4 CPU threads. Each configuration gets 50 warm-up calls followed by 1,000 timed calls, and the whole grid is repeated 10 times in a freshly shuffled order so that run order and cache state cannot result in bias towards any model. We report the median of the per-repeat medians, with a 95% interval across repeats. MACs are counted with forward hooks on every linear layer. Dispatched operations are counted by crossed PyTorch’s `__torch_function__` protocol during one forward pass, and the kernels those operations execute are counted with `torch.profiler`. We also count `nn.Module` calls with forward hooks. These sweep timings are separate from the batch-1,000 inference times in Table III, which are measured on the trained models inside the training study; the two agree where they intersect.

F. Environment

All main-study and sweep measurements run on a single machine (Apple M5 CPU, macOS, Python 3.9, PyTorch 2.8.0) with `torch.set_num_threads(1)` unless stated otherwise, so the timings compare architectures rather than thread scheduling. No GPU is used:

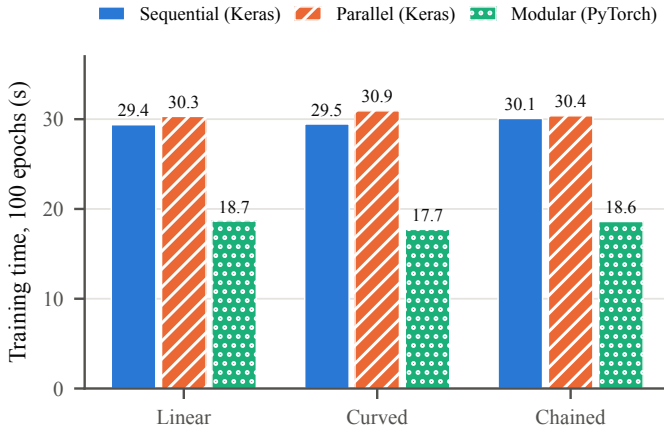


Fig. 3. Pilot (Benchmark 1) training time. The Modular network appears $\sim 1.62\times$ faster, but it was the only model implemented in PyTorch, and it is functionally identical to Sequential. The gap measures framework overhead, not architecture.

TABLE II
PILOT (BENCHMARK 1) RESULTS AS ORIGINALLY REPORTED: TRAINING TIME (S) AND FINAL *training* MSE, SINGLE SEED. COMPARE THE MSEs WITH THE TARGET VARIANCES: 34.7, 228.8, AND 10.0.

Model	Linear		Curved		Chained	
	time	MSE	time	MSE	time	MSE
Sequential (Keras)	29.4	37.80	29.5	239.64	30.1	10.39
Parallel (Keras)	30.3	37.83	30.9	240.53	30.4	10.44
Modular (PyTorch)	18.7	38.16	17.7	245.68	18.6	10.55

at this model scale, kernel-launch overhead would dominate against any GPU measurement [10]. All code and raw results are available at <https://github.com/aryanamol10/Research-Paper-Comparative-Analysis-of-Neural-Models>.

IV. PILOT STUDY: HOW A BENCHMARK CAN MISLEAD

Before the main study, we ran a pilot benchmark (Benchmark 1) of the three single-paradigm models in a Google Colab notebook. Its headline result was noticeable: the Modular network trained about $1.62\times$ faster than Sequential and Parallel on every task (Fig. 3, Table II), and its accuracy was only a fraction worse. Taken at face value, this suggests that modularity is a free latency win. Auditing the pilot showed that the result has nothing to do with architecture. We report the audit because each flaw is common in small-scale benchmarking [12], [33].

Confound 1: framework, not architecture. Sequential and Parallel were built in Keras/TensorFlow [37], [38] using `model.fit`, while Modular was a hand-written PyTorch loop. Modular and Sequential compute the same 1–128–1 function, so the ~ 11 s gap is entirely framework and training-loop overhead. Framework-level differences of this size are well documented [11]. They show that any timing comparison across frameworks does not yield much about architecture.

Confound 2: uninformative inputs. The pilot trained on $x \sim \mathcal{U}(0, 1)$ drawn independently of the targets, which were

computed on the evenly spaced grid. With no information in the inputs, the best any model can do is predict the target mean, which gives an MSE equal to the target variance. The reported losses (37.8, 240, 10.4) sit just above those variances (34.7, 228.8, 10.0). In other words, none of the pilot models learned the task, so their accuracy ranking is noise.

Confound 3: no held-out data, one seed. Losses were final-epoch *training* losses from a single seed with shuffling disabled, so neither generalization nor running variance could be assessed.

The main study (Section III) fixes all three: one framework and one training loop, true inputs, a held-out split, and five seeds. The identical Sequential/Modular pair stays in as a built-in check. If the protocol is sound, the two must agree on accuracy and differ only by noise.

V. RESULTS

Table III reports accuracy and cost for every (model, task) pair, Fig. 4 shows test-error trajectories and behavior, and Fig. 5 places each model on an accuracy-cost plane. The latency sweep is shown in Figs. 6 and 7.

A. The Control Pair: Identical Math, Different Code

Sequential and Modular compute the same function and, under the same seed, produce the same test MSE on every task (Table III). Their training step times differ by only 1%, confirming that the pilot’s $1.62\times$ “Modular advantage” (Section IV) disappears once the framework and training loop are held constant. Their batch-1 inference latency, however, differs by 17% (11.7 vs. 10.0 μ s), with non-overlapping 95% intervals across ten shuffled repeats, so the gap is not run-order noise. Both dispatch three tensor operations, but Sequential’s `nn.Sequential` container makes 5 `nn.Module` calls per forward pass against Modular’s 3. The control pair therefore isolates a second, purely software overhead, telling us how the code organization costs time even when the math is the same (Section V-C).

B. Accuracy: Structure Pays Off Only When the Task Has Structure

Linear and curved tasks. Every model reaches the noise floor on the linear task ($\text{MSE} \approx 1$, $R^2 = 0.971$), and on the curved task every model finishes within 2% of the floor of 4. Simple targets leave nothing for topology to improve upon, so at a 385-parameter budget the arrangement of parameters does not change the final accuracy. It does change *how fast* the floor is reached. On the curved task H-B converges in 8 epochs versus 69 for Sequential, about $5.3\times$ less training time to convergence (0.45 s vs. 2.37 s) despite its higher cost per step.

Narrow trunks are fragile. H-C’s trunk width also matters for reliability. In a preliminary configuration with a 17–17 trunk (376 parameters), one of five curved-task seeds settled in a poor solution ($\text{MSE} 21.4$ vs. ≈ 4.0 for the rest). The exact-budget 20–15 trunk used here trains reliably on every seed, but it still has the largest spread on the curved task.

TABLE III

MAIN RESULTS (MEAN $\pm$ STD OVER 5 SEEDS; HELD-OUT TEST SET OF 1,000 POINTS). IRREDUCIBLE TEST MSE IS 1, 4, AND 0.25 FOR THE LINEAR, CURVED, AND CHAINED TASKS. BOLD = BEST AT SHOWN PRECISION. [†]WELCH'S t -TEST VS. SEQUENTIAL, $p < 0.05$. INFERENCE = FORWARD PASS OVER THE WHOLE TEST SET (BATCH 1,000), SINGLE CPU THREAD.

Model	Test MSE $\downarrow$	$R^2 \uparrow$	Train (ms/step) $\downarrow$	Inference (ms) $\downarrow$	Converge (epoch) $\downarrow$
<i>Linear task</i>					
Sequential	1.000 $\pm$ 0.001	0.971	0.275 $\pm$ 0.002	0.082 $\pm$ 0.000	7 $\pm$ 1
Parallel	1.000 $\pm$ 0.002	0.971	0.325 $\pm$ 0.004	0.091 $\pm$ 0.000	7 $\pm$ 0
Modular	1.000 $\pm$ 0.001	0.971	0.271 $\pm$ 0.000	0.078 $\pm$ 0.000	7 $\pm$ 1
H-A	1.001 $\pm$ 0.002	0.971	0.418 $\pm$ 0.003	0.094 $\pm$ 0.001	8 $\pm$ 1
H-B	1.019 $\pm$ 0.016	0.970	0.481 $\pm$ 0.008	0.130 $\pm$ 0.001	7 $\pm$ 2
H-C	1.000 $\pm$ 0.002	0.971	0.375 $\pm$ 0.000	0.065 $\pm$ 0.001	12 $\pm$ 3
<i>Curved task</i>					
Sequential	4.062 $\pm$ 0.045	0.982	0.274 $\pm$ 0.005	0.081 $\pm$ 0.000	69 $\pm$ 9
Parallel	4.033 $\pm$ 0.037	0.982	0.321 $\pm$ 0.001	0.093 $\pm$ 0.002	69 $\pm$ 2
Modular	4.062 $\pm$ 0.045	0.982	0.267 $\pm$ 0.001	0.078 $\pm$ 0.000	69 $\pm$ 9
H-A	4.007 $\pm$ 0.024	0.983	0.416 $\pm$ 0.003	0.094 $\pm$ 0.000	24 $\pm$ 4
H-B	4.005 $\pm$ 0.049	0.983	0.474 $\pm$ 0.004	0.130 $\pm$ 0.001	8 $\pm$ 1
H-C	4.078 $\pm$ 0.146	0.982	0.375 $\pm$ 0.005	0.065 $\pm$ 0.000	36 $\pm$ 10
<i>Chained task</i>					
Sequential	0.571 $\pm$ 0.118	0.943	0.271 $\pm$ 0.001	0.082 $\pm$ 0.001	90 $\pm$ 4
Parallel	0.645 $\pm$ 0.159	0.935	0.328 $\pm$ 0.017	0.092 $\pm$ 0.001	89 $\pm$ 5
Modular	0.571 $\pm$ 0.118	0.943	0.272 $\pm$ 0.006	0.079 $\pm$ 0.001	90 $\pm$ 4
H-A	0.371 $\pm$ 0.022 [†]	0.963	0.415 $\pm$ 0.011	0.094 $\pm$ 0.001	74 $\pm$ 6
H-B	0.326 $\pm$ 0.019[†]	0.967	0.476 $\pm$ 0.006	0.130 $\pm$ 0.003	61 $\pm$ 19
H-C	0.415 $\pm$ 0.029	0.958	0.370 $\pm$ 0.002	0.066 $\pm$ 0.000	66 $\pm$ 19

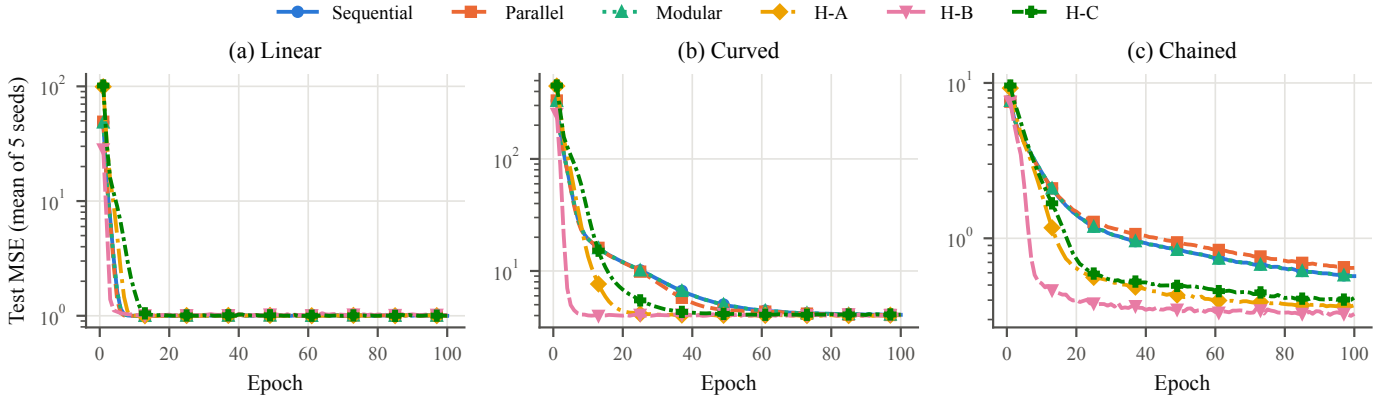


Fig. 4. Test MSE per epoch (log scale, mean of five seeds). Hybrids reach low error in far fewer epochs on the curved and chained tasks. H-B (pink) is fastest to finish and ends lowest on the chained task.

Chained task. Here topology matters. H-B, in which the modules mirror the three stages of the target, reaches 0.326 test MSE, 43% below Sequential (0.571; $p < 0.05$). H-A, which has no knowledge of the stages, still improves by 35% (0.371; $p < 0.05$). Measured against the noise floor of 0.25, H-B's excess error (0.076) is $4.2\times$ smaller than Sequential's (0.321). Both hybrids also have much smaller variance (± 0.02 vs. ± 0.12). The single-paradigm models are still improving at epoch 100 (Fig. 4c), so part of their gap is slower convergence rather than a lower ceiling. At a fixed budget, the gap is what a practitioner would get. Plain breadth does not help: Parallel is the worst model on the chained task (0.645).

C. Latency: Operator Count, Not Parameter Count

Table I and Fig. 6 show that, at a nearly constant parameter budget, single inference latency spans almost $4\times$, from 10.0 μ s

(Modular) to 38.2 μ s (H-B). Parameter count cannot explain this: all models have 385–387 parameters. Neither can MACs: Parallel and Sequential have the same 256 MACs, but Parallel is slower, and H-C has the most MACs (350) but is faster than H-A and H-B. What does explain it is the number of tensor operations dispatched per forward call. A linear fit of batch-1 latency on operation count gives $R^2 = 0.98$ with a slope of about 2.8 μ s per operation. At this scale, each Linear, ReLU, cat, or add pays a constant framework transmission cost that lessens its arithmetic. H-B has 13 operations and runs $3.3\times$ slower than Sequential. Counting the ATen kernels those operations execute gives an equally good fit ($R^2 = 0.99$), so the effect is not an artifact of how we count. Adding the number of `nn.Module` calls as a second predictor explains almost all remaining variance ($R^2 = 0.998$; roughly 1.8 μ s per operation

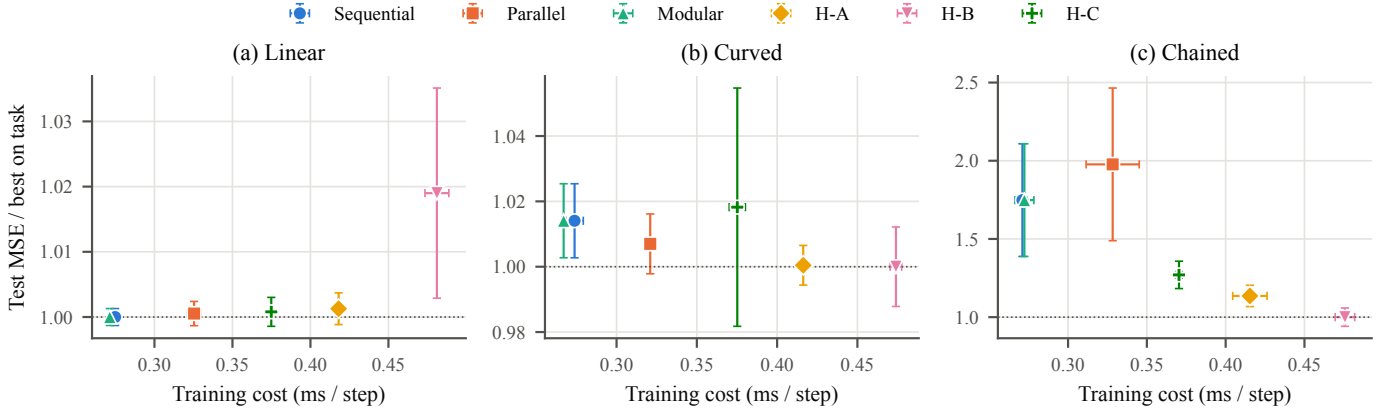


Fig. 5. Accuracy vs. training cost. The y -axis is test MSE divided by the best model’s MSE on that task ($1 = \text{best}$). Error bars are ± 1 std over the seeds. On simple tasks all models tie on accuracy, so the cheapest model wins. On the chained task, accuracy can be bought with per-step cost.

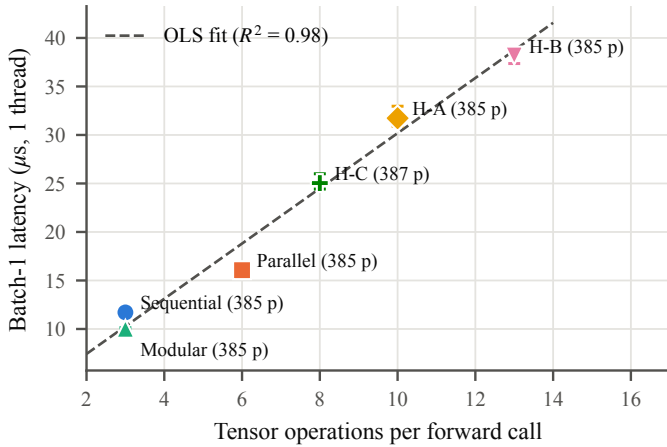


Fig. 6. Batch-1 inference latency vs. tensor operations per forward call. Parameter counts are in parentheses; error bars are 95% intervals across ten shuffled repeats. Latency is almost perfectly explained by the operator count ($R^2 = 0.98$) and is unrelated to the actual parameter count.

and $1.0\mu\text{s}$ per module call), including the Sequential/Modular gap. With six architectures and three fitted coefficients, we treat this decomposition as being informative rather than predictive.

Batch size changes the ranking. Figure 7a shows two regimes. Up to a batch of about 64, latency is nearly flat and ordered by operator count. Past roughly 500 samples, per-element work takes over and the ordering reshuffles. At batch 4096, H-C becomes the *fastest* model ($156\mu\text{s}$ vs. $196\mu\text{s}$ for Sequential) despite having the most MACs. This is consistent with memory traffic having more value than arithmetic: H-C’s widest activation is 20 values per sample, while Sequential’s is 128 and Parallel’s concatenation writes 128 more. The two models that build intermediate concatenated tensors are the slowest at batch 4096: H-B ($322\mu\text{s}$) and Parallel ($314\mu\text{s}$). The latency ranking of these architectures, for this reason, depends on the batch size at of which it is being measured.

Threads. With four occurring threads (Fig. 7b), batch-1 latency is unchanged. At larger batches the effect is not the

same. Sequential improves at batch 4096 (196 to $146\mu\text{s}$), but Parallel at batch 1,000 slows from 90 to $179\mu\text{s}$ and H-A at batch 4096 slows to $352\mu\text{s}$. For below kilobyte models, thread synchronization overhead can go over the parallel work.

VI. DISCUSSION

A. Which Architecture Should a Practitioner Pick?

The results support a simple decision rule for small models on latency-sensitive paths.

- 1) *If the target has no known structure, keep it shallow and minimize operators.* On the linear and curved tasks every topology reached the noise floor, so the cheapest one, a single wide hidden layer with three operations, becomes the best choice. It trains fastest per step and has the lowest batch-1 latency.
- 2) *If the target is compositional and its stages are known, a staged modular hybrid is worth its overhead.* H-B cut chained-task excess error by $4.2\times$ at $1.76\times$ the training cost per step and $3.3\times$ the batch-1 latency. For offline or batched inference, that trade is attractive. For per-sample loops, this is not the case.
- 3) *Measure latency at the deployment batch size.* The ranking at batch 1 (operator-bound) and at batch 4096 (memory-bound) differ. A benchmark at one batch size can recommend the incorrect model for another.

B. Hybrids Converge Faster

A less expected result is how much faster the hybrids converge. H-B reached the curved-task noise floor in 8 epochs, and H-A in 24, against 69 for single models. We hypothesize that the additive multi-path output acts like a remaining groupage [18], [23], where several sub-networks fit different parts of the target at once, so no single narrow path is required to carry the whole function. When training compute rather than inference latency is the bottleneck, this can outweigh the hybrids’ higher per-step cost.

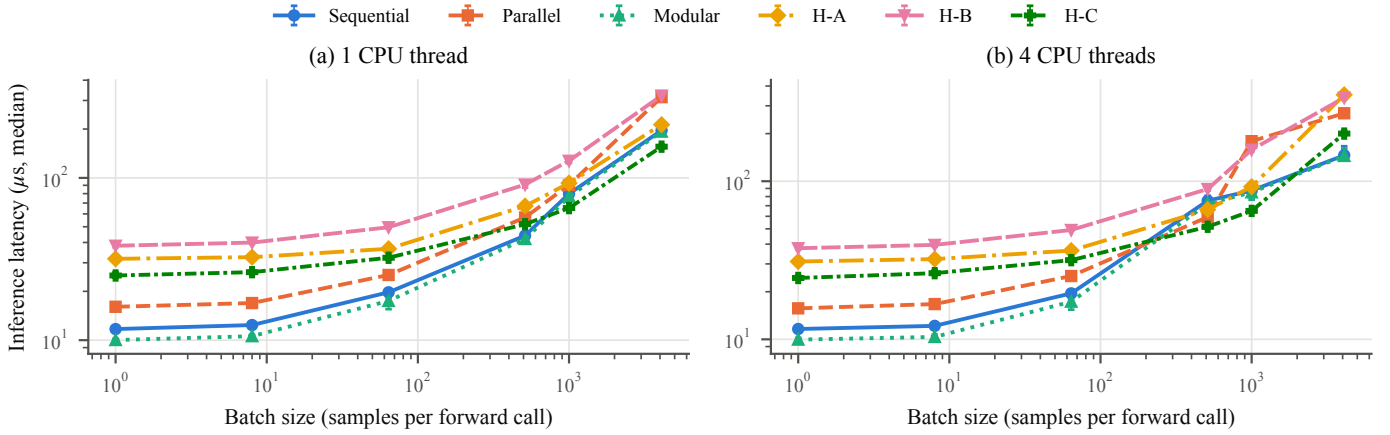


Fig. 7. Inference latency vs. batch size (median over 10 shuffled repeats of 1,000 calls; bars show the 10th–90th percentile of individual calls). (a) With one thread, the curves are flat up to batch ≈ 64 , where overhead dominates, and the ranking follows operator count. By batch 4096 it inverts toward the narrow-activation H-C. (b) Four threads rarely help models this small and sometimes hurt (Parallel at batch 1,000).

C. Lessons for Small-Scale Benchmarking

The pilot study shows how easily a benchmark can produce a confident, wrong conclusion. Three practices caught it and are cheap enough for any student or practitioner to conduct. First, include a *null control*: a pair that should show no difference, such as our identical Sequential and Modular networks. Our benchmarking served multiple roles, exposing the pilot’s framework confound, catching CPU contention during one training pass, and revealing module-call overhead that operation counts alone miss. Second, hold the framework and training loop fixed so that timing measures the model and not the software stack [11]. Third, check accuracy against small baselines. A model’s output that gives the loss being equal to the target variance has not learned anything [33].

D. Limitations and Threats to Validity

Scale. The models are tiny (≈ 385 parameters) and the tasks shown for testing are one-dimensional. This is done deliberately, as it isolates the topology from overall capacity and makes the five-seed run cheap. It also means the total latencies are dominated by per-operator overhead in a way that will shrink, though not vanish, for larger models, where compute and traffic take over [10]. The ranking of architectures at batch 4096, where arithmetic begins to matter, is a better guide to larger-scale behavior than batch 1.

Hardware and runtime. All timings come from one Apple M5 CPU running eager-mode PyTorch. Graph compilers (TorchScript, `torch.compile`, ONNX Runtime) fuse small operators and would likely create narrowing in the count penalty we observe. GPUs and microcontrollers would have very different overhead profiles. We therefore present latency *trends* across architectures, not total numbers to be quoted for other platforms. [TODO: optional: rerun latency_sweep.py with torch.compile or on a Raspberry Pi / Colab GPU for a second hardware point]

Task design. The chained task was written by the same author who designed H-B, so H-B’s structural match to it is

an intentional best case for modular inductive bias, not an unbiased test. We do not claim that staged modularity helps on unknown compositional problems, only that it *can* when the decomposition is known.

Statistics. Five seeds detect large effects but not small ones. We report Welch’s *t*-tests without multiple-comparison correction, so single and marginal results should be read as suggestive. Hyperparameters (learning rate, epochs, batch size) were fixed for all models rather than tuned per architecture, which may disadvantage architectures that give preference for other settings.

VII. CONCLUSION

We benchmarked six neural architectures that share a ≈ 385 -parameter budget and differ only in how those parameters are arranged. Three findings stand out. First, configuration changes final accuracy only when the target has structure for it to match. A staged modular hybrid reduced error on a compositional task by 43%, while every other structure tied on simple tasks. Second, hybrids complete in far fewer epochs, which can offset their higher cost given per step. Third, small-model latency is decided by operator count at small batch sizes and by activation width at large ones, and parameter count predicts neither. A pilot study that reached the opposite conclusion through a framework perplexities shows why these comparisons need null controls and single software stacks.

Future work will extend the study to compiled runtimes (`torch.compile`, ONNX Runtime) that combine small operators, to microcontroller and GPU targets, to bigger parameter budgets where arithmetic dominates overhead, and to real-world tasks where the stage partitioning must be learned rather than designed.

ACKNOWLEDGMENT

The pilot study used Google Colab. AI tools assisted with experiment code, figures, and drafting; the author verified all content.

REFERENCES

- [1] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9.
- [2] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 5987–5995.
- [3] R. Caruana, "Multitask learning," *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997.
- [4] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017, arXiv:1701.06538.
- [5] R. Eldan and O. Shamir, "The power of depth for feedforward neural networks," in *Proceedings of the 29th Conference on Learning Theory (COLT)*, ser. Proceedings of Machine Learning Research, vol. 49, 2016, pp. 907–940, arXiv:1512.03965.
- [6] Z. Lu, H. Pu, F. Wang, Z. Hu, and L. Wang, "The expressive power of neural networks: A view from the width," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, arXiv:1709.02540.
- [7] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," in *Computer Vision – ECCV 2018*, ser. Lecture Notes in Computer Science. Springer, 2018, pp. 122–138.
- [8] M. Dehghani, A. Arnab, L. Beyer, A. Vaswani, and Y. Tay, "The efficiency misnomer," in *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022, arXiv:2110.12894.
- [9] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau, U. Thakker, A. Torrini, P. Warden, J. Cordaro, G. Di Guglielmo, J. Duarte, S. Gibellini, V. Parekh, H. Tran, N. Tran, N. Wenxu, and X. Xuesong, "MLPerf tiny benchmark," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021, arXiv:2106.07597.
- [10] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [11] S. Shi, Q. Wang, P. Xu, and X. Chu, "Benchmarking state-of-the-art deep learning software tools," in *Proceedings of the 7th International Conference on Cloud Computing and Big Data (CCBD)*, 2016, pp. 99–104.
- [12] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Sepah, E. Raff, K. Madan, V. Voleti, S. E. Kahou, V. Michalski, D. Serdyuk, T. Arbel, C. Pal, G. Varoquaux, and P. Vincent, "Accounting for variance in machine learning benchmarks," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 747–769.
- [13] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.
- [14] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.
- [15] M. Telgarsky, "Benefits of depth in neural networks," in *Proceedings of the 29th Conference on Learning Theory (COLT)*, ser. Proceedings of Machine Learning Research, vol. 49, 2016, pp. 1517–1539, arXiv:1602.04485.
- [16] S. Zagoruyko and N. Komodakis, "Wide residual networks," in *Proceedings of the British Machine Vision Conference (BMVC)*, 2016, pp. 87.1–87.12.
- [17] S. Ruder, "An overview of multi-task learning in deep neural networks," arXiv preprint arXiv:1706.05098, 2017.
- [18] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, arXiv:1612.01474.
- [19] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [20] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein, "Neural module networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 39–48.
- [21] J. Pfeiffer, S. Ruder, I. Vulić, and E. M. Ponti, "Modular deep learning," *Transactions on Machine Learning Research*, 2023, arXiv:2302.11529.
- [22] D. Hupkes, V. Dankers, M. Mul, and E. Bruni, "Compositionality decomposed: How do neural networks generalise?" *Journal of Artificial Intelligence Research*, vol. 67, pp. 757–795, 2020.
- [23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [24] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2261–2269.
- [25] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "MnasNet: Platform-aware neural architecture search for mobile," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2815–2823.
- [26] H. Cai, L. Zhu, and S. Han, "ProxylessNAS: Direct neural architecture search on target task and hardware," in *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019, arXiv:1812.00332.
- [27] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *Journal of Machine Learning Research*, vol. 20, no. 55, pp. 1–21, 2019.
- [28] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. St. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, "MLPerf inference benchmark," in *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 446–459.
- [29] C. Coleman, D. Kang, D. Narayanan, L. Nardi, T. Zhao, J. Zhang, P. Bailis, K. Olukotun, C. Ré, and M. Zaharia, "Analysis of DAWN-Bench, a time-to-accuracy machine learning performance benchmark," *ACM SIGOPS Operating Systems Review*, vol. 53, no. 1, pp. 14–25, 2019.
- [30] A. Canziani, A. Paszke, and E. Culurciello, "An analysis of deep neural network models for practical applications," arXiv preprint arXiv:1605.07678, 2016.
- [31] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv preprint arXiv:1704.04861, 2017.
- [32] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [33] Z. C. Lipton and J. Steinhardt, "Troubling trends in machine learning scholarship," *Queue*, vol. 17, no. 1, pp. 45–77, 2019.
- [34] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010, pp. 807–814.
- [35] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, arXiv:1912.01703.
- [36] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015, arXiv:1412.6980.
- [37] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: A system for large-scale machine learning," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.
- [38] F. Chollet et al., "Keras," <https://keras.io>, 2015.